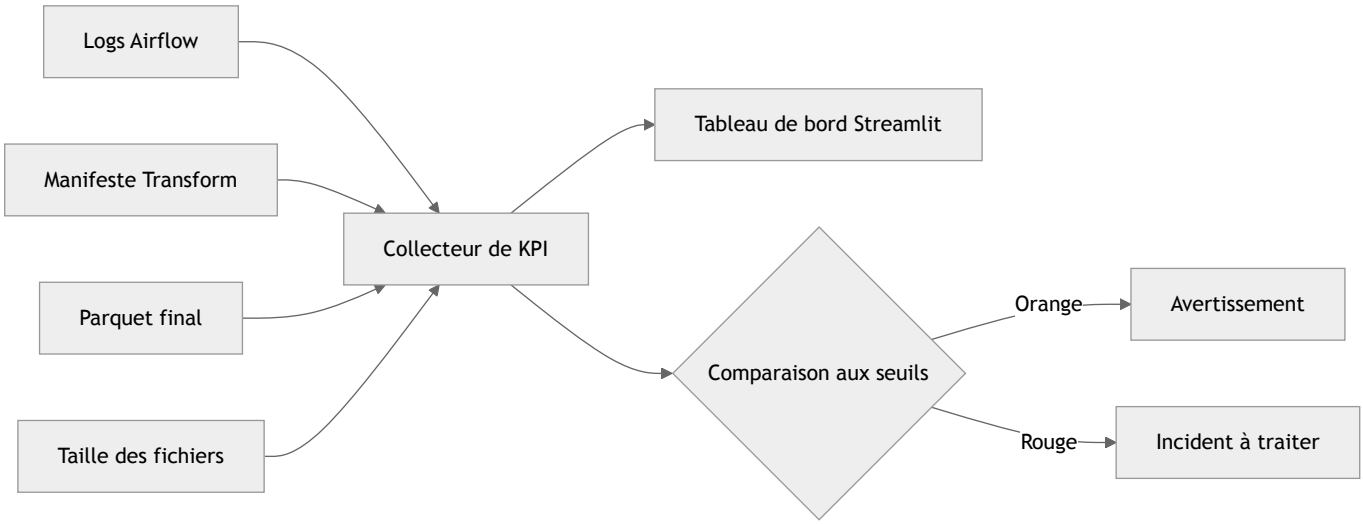
Étape 05 — Plan de monitoring du pipeline ETL

1. Objectif

Le monitoring doit répondre rapidement à trois questions :

1. Les données produites sont-elles fiables et bien associées à leurs images ?
2. Le pipeline s'exécute-t-il correctement et dans un délai acceptable ?
3. Le volume traité reste-t-il compatible avec les ressources disponibles ?

Le dispositif s'appuie sur les éléments déjà produits par le pipeline : manifeste du run courant, Parquet incrémental, logs Airflow et tailles des fichiers. Le tableau de bord est disponible dans `dashboard/etl_kpi_dashboard.py`.



2. Indicateurs et seuils

Les seuils sont centralisés dans `config/monitoring.json` afin de pouvoir les modifier sans changer le code du tableau de bord.

Axe	KPI	Calcul	Cible	Avertissement	Critique
Qualité	Taux de validité	entrées validées / entrées brutes	≥ 98 %	< 98 %	< 95 %
Qualité	Complétude multimodale	lignes avec texte et preuve d'image / lignes du run	≥ 99 %	< 99 %	< 97 %
Qualité	Doublons retirés	occurrences dédupliquées dans le run	information	hausse inhabituelle	à analyser selon la source
Qualité	Couverture des labels	lignes ayant un label source / lignes du run	information	aucune	aucune
Fiabilité	Taux de succès	runs réussis / runs observés	≥ 95 %	< 95 %	< 90 %
Rapidité	Durée du run	dernière validation − première extraction	< 60 s	≥ 60 s	≥ 120 s
Fraîcheur	Âge du dernier run	maintenant − fin du dernier run	< 30 min	≥ 30 min	≥ 60 min
Coût	Coût calcul estimé	durée × tarif horaire	tendance stable	hausse de 30 %	hausse durable à analyser
Coût	Coût stockage estimé	Go suivis × tarif mensuel	tendance stable	hausse de 30 %	espace disponible faible

Un doublon n'est pas une donnée invalide : il indique que les lots du run contiennent la même publication. Le dédoublonnage traite le run courant ; l'upsert PostgreSQL gère les publications déjà présentes dans l'historique. Une hausse brutale peut toutefois révéler une pagination mal configurée.

Les coûts affichés sont des estimations paramétrables. Ils servent à comparer les runs ; ils ne remplacent pas la facture d'un fournisseur cloud ou d'une API.

3. Sources de mesure

Qualité des données

- `data/processed/transformation_manifest.json` : chemins exacts, volumes bruts, validés, rejetés et dédupliqués du dernier run ;
- `data/processed/publications.parquet` : contrôle des champs texte, image, hash et taille du lot courant ;
- `data/rejected/invalid_records.jsonl` : motif de chaque rejet.

Exécution

- `airflow/logs/dag_id=checkitai_multimodal_etl/` : début, fin, tentative et résultat de chaque tâche ;
- interface Airflow : état du DAG, graphe des tâches et relances ;
- XCom de `validate_postgres` : nombre de lignes, associations invalides et doublons de clé primaire.

Ressources et coûts

- taille cumulée de `data/raw`, `data/processed`, `data/images` et `airflow/logs` ;
- durée réelle du dernier run ;
- tarifs de calcul et de stockage saisis dans la barre latérale Streamlit.

4. Journalisation

Journal	Contenu	Consultation	Conservation proposée
Logs de tâche Airflow	exécution, erreurs, retries, valeur retournée	après chaque run et incident	30 jours
Logs d'extraction	appels, images téléchargées, erreurs HTTP	en cas de baisse de volume	30 jours
Log de transformation	contrôles, rejets, dédoublonnage	après chaque run	90 jours
Manifeste	paramètres, hashes, compteurs et version du schéma	audit et comparaison	12 mois
Rejets JSONL	ligne rejetée et motif	investigation qualité	90 jours

Les logs ne doivent contenir ni clé API, ni mot de passe, ni URL de connexion complète. `.env` et `.env.airflow` restent hors Git avec des permissions `0600`.

5. Fréquence des contrôles

Moment	Contrôle	Responsable
À chaque tâche	état, exception et retry automatiques Airflow	Airflow
À chaque fin de DAG	contrôle PostgreSQL et association multimodale	tâche `validate_postgres`
Après chaque run	consultation du bandeau de santé et des KPI	exploitant du pipeline
Toutes les 15 minutes	exécution automatique du DAG	scheduler Airflow
Chaque heure	fraîcheur, taux de succès et durée	exploitant du pipeline
Chaque semaine	évolution des doublons, volumes et durées par source	référent Data
Chaque mois	coût estimé, occupation disque et ajustement des seuils	référent Data

`CHECKITAI_SCHEDULE=*/15 * * * *` déclenche automatiquement le DAG à chaque quart d'heure tant que le scheduler fonctionne. Une absence de succès pendant 30 minutes déclenche un avertissement ; après 60 minutes, l'état devient critique.

6. Alertes

Niveau vert

Tous les KPI respectent leur cible. Aucune action n'est requise.

Niveau orange

Un seuil d'avertissement est franchi. L'exploitant consulte le tableau de bord et les logs dans la journée, puis compare avec le run précédent. Aucun retraitement n'est lancé tant que la cause n'est pas comprise.

Niveau rouge

Un run échoue, les données valides passent sous 95 %, la complétude multimodale sous 97 %, la durée dépasse 120 secondes ou le dernier succès date de plus de 60 minutes. L'incident doit être traité avant de considérer les nouvelles données comme disponibles.

Dans l'environnement actuel, l'état est visible dans Airflow et Streamlit. Lors d'un déploiement partagé, un `on_failure_callback` Airflow pourra envoyer une notification vers le canal choisi. Les identifiants de notification devront être stockés dans une connexion Airflow chiffrée, jamais dans le DAG.

7. Procédure de gestion d'un incident

1. Identifier la première tâche rouge dans la vue Graph Airflow.
2. Lire son dernier log et noter le code HTTP, l'exception ou le contrôle en échec.
3. Vérifier si la cause est temporaire : quota API, indisponibilité réseau ou source distante.
4. Pour un défaut de données, consulter `invalid_records.jsonl` et le manifeste ; ne pas désactiver les contrôles pour faire passer le run.
5. Corriger la configuration ou le code, puis relancer uniquement après avoir vérifié que les données brutes sont cohérentes.
6. Confirmer dans Streamlit le retour au vert et documenter la cause.

Les retries Airflow absorbent déjà les erreurs temporaires : deux nouvelles tentatives espacées de deux minutes. Une erreur persistante reste visible comme un échec et bloque les tâches dépendantes.

8. Lancer le tableau de bord

```
uv sync
uv run streamlit run dashboard/etl_kpi_dashboard.py
```

Le tableau de bord s'ouvre par défaut sur http://localhost:8501. Les KPI de qualité et les graphiques par source décrivent le lot transformé courant ; l'historique des runs et le stockage restent cumulatifs. Le bouton **Actualiser les données** vide le cache de 30 secondes et relit les sorties.

Un instantané JSON peut également être produit sans interface :

```
uv run python scripts/collect_kpis.py --output data/monitoring/kpi_snapshot.json
```

Les options `--manifest`, `--parquet`, `--airflow-logs` et `--data-dir` permettent de recalculer les mêmes indicateurs sur un autre jeu de sorties sans modifier le code.